LatticeTree: Ultra-Fast SNARK prover for large circuits

Daniel Rubin, Simon Judd, John Wu, Vlad Heintz

January 13, 2026

Abstract

Current zkVMs cannot prove high-throughput parallel systems in real-time - they're architecturally limited to sequential processing. We're building a new class of SNARK prover with four breakthroughs: tree-style parallel folding for distributed proving, customizable constraint systems with advanced lookup tables, non-uniform proof-carrying data for cross-relation folding, and lattice-based post-quantum cryptography. This enables sub-second proving for systems processing hundreds of thousands of daily blocks, unlocking real-time verification for the highest-performance blockchains, cross-chain bridges, and financial applications that need millisecond-scale cryptographic guarantees.

1 Introduction

Zero-knowledge virtual machines (zkVMs) have emerged as a critical infrastructure component for verifiable computation, enabling applications from blockchain scalability to privacy-preserving financial systems. However, existing zkVM architectures face fundamental limitations when applied to high-throughput parallel systems that process hundreds of thousands of transactions daily.

The core challenge lies in the sequential nature of current SNARK proving systems. While modern blockchains and distributed systems achieve massive throughput through parallel execution, zkVMs remain bottlenecked by single-threaded proof generation. This architectural mismatch creates a proving time that scales linearly with system throughput, making real-time verification impossible for high-performance applications.

Recent advances in recursive SNARKs [KS23] and folding schemes [ZGGX23, NDC+24] have shown promising results for proof composition and aggregation. The development of customizable constraint systems [STW23] and improved lookup table techniques [PH23] have also contributed to enhanced SNARK efficiency. However, these improvements still operate within the fundamental constraint of sequential proof generation.

We present LatticeTree, a novel SNARK prover architecture that breaks through these limitations with four key innovations. First, we introduce tree-style parallel folding that enables distributed proof generation across multiple nodes. Second, we implement customizable constraint systems with advanced lookup tables optimized for high-throughput verification. Third, we develop non-uniform proof-carrying data (PCD) techniques that allow efficient folding across different constraint relations. Finally, we integrate lattice-based cryptographic primitives to provide post-quantum security guarantees.

Our approach enables sub-second proving times for systems processing hundreds of thousands of daily operations, representing a fundamental shift from sequential to parallel proof generation. This breakthrough unlocks real-time cryptographic verification for applications requiring millisecond-scale guarantees, including high-performance blockchains, cross-chain bridges, and latency-sensitive financial systems.

2 Background and Notation

Definition 2.1 (Multilinear Extensions [STW23]) *An ℓ-variate polynomial $p : \mathbb{F}^\ell \to \mathbb{F}$ is said to be **multilinear** if p has degree at most one in each variable. Let $f : \{0,1\}^\ell \to \mathbb{F}$ be any function mapping the ℓ-dimensional Boolean hypercube to a field $\mathbb{F}$. A polynomial $g : \mathbb{F}^\ell \to \mathbb{F}$ is said to **extend** f if $g(x) = f(x)$ for all $x \in \{0,1\}^\ell$.*

For any $f : \{0,1\}^\ell \to \mathbb{F}$, there is a unique multilinear polynomial $\tilde{f} : \mathbb{F}^\ell \to \mathbb{F}$ that extends f. The polynomial $\tilde{f}$ is referred to as the ***multilinear extension (MLE)*** *of f.*

A particular multilinear extension that arises frequently in the design of interactive proofs is $\widetilde{eq}$, which is the MLE of the function $eq : \{0,1\}^s \times \{0,1\}^s \to \mathbb{F}$ defined as:

$$eq(x,e) = \begin{cases} 1 & \text{if } x = e \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

An explicit expression for $\widetilde{eq}$ is:

$$\widetilde{eq}(x,e) = \prod_{i=1}^{s} (x_i e_i + (1-x_i)(1-e_i)). \tag{2}$$

2.1 Customizable Constraint Systems

Definition 2.2 (CCS [KS23]) *Consider size bounds $m, n, N, \ell, t, q, d \in \mathbb{N}$ where $n > \ell$. Let $s = \log m$ and $s' = \log n$. We define the customizable constraint system (CCS) relation, $\mathcal{R}_{CCS}$, over structure, instance, witness tuples as follows.*

An $\mathcal{R}_{CCS}$ structure $\mathcal{S}$ consists of:

- *a sequence of sparse multilinear polynomials in $s + s'$ variables $\tilde{f}_{M_1}, \ldots, \tilde{f}_{M_t}$ such that they evaluate to a non-zero value in at most $N = \Omega(m)$ locations over the Boolean hypercube $\{0,1\}^s \times \{0,1\}^{s'}$;*
- *a sequence of q multisets $[S_1, \ldots, S_q]$, where an element in each multiset is from the domain $\{1, \ldots, t\}$ and the cardinality of each multiset is at most d;*
- *a sequence of q constants $[c_1, \ldots, c_q]$, where each constant is from $\mathbb{F}$.*

An $\mathcal{R}_{CCS}$ instance consists of public input and output vector $x \in \mathbb{F}^\ell$. An $\mathcal{R}_{CCS}$ witness consists of a multilinear polynomial $\tilde{w}$ in $s' - 1$ variables. We have that $(\mathcal{S}, x, \tilde{w}) \in \mathcal{R}_{CCS}$ if and only if for all $x \in \{0,1\}^s$,

$$\sum_{i=1}^{q} c_i \cdot \prod_{j \in S_i} \left(\sum_{y \in \{0,1\}^{\log m}} \tilde{f}_{M_j}(x,y) \cdot \tilde{z}(y) \right) = 0, \tag{3}$$

where $\tilde{z}$ is an s'-variate multilinear polynomial such that $\tilde{z}(x) = \widetilde{(w,1,x)}(x)$ for all $x \in \{0,1\}^{s'}$.

Definition 2.3 (Linearized CCS) *Consider size bounds $m, n, N, \ell, t, q, d \in \mathbb{N}$ where $n = 2 \cdot (\ell + 1)$. Let $s = \log m$ and $s' = \log n$. We define the linearized committed customizable constraint system (LCCS) relation, $\mathcal{R}_{LCCS}$, over structure, instance, witness tuples as follows.*

An $\mathcal{R}_{LCCS}$ structure $\mathcal{S}$ consists of:

- *a sequence of sparse multilinear polynomials in $s + s'$ variables $\tilde{f}_{M_1}, \ldots, \tilde{f}_{M_t}$ such that they evaluate to a non-zero value in at most $N = \Omega(m)$ locations over the Boolean hypercube $\{0,1\}^s \times \{0,1\}^{s'}$;*
- *a sequence of q multisets $[S_1, \ldots, S_q]$, where an element in each multiset is from the domain $\{1, \ldots, t\}$ and the cardinality of each multiset is at most d;*
- *a sequence of q constants $[c_1, \ldots, c_q]$, where each constant is from $\mathbb{F}$.*

An $\mathcal{R}_{LCCS}$ instance is a tuple $(u, x, r, v_1, \ldots, v_t) \in (\mathbb{F}, \mathbb{F}^\ell, \mathbb{F}, \mathbb{F}^t)$. An $\mathcal{R}_{LCCS}$ witness consists of a multilinear polynomial $\tilde{w}$ in $s' - 1$ variables. We have that $(\mathcal{S}, (u, x, r, v_1, \ldots, v_t), \tilde{w}) \in \mathcal{R}_{LCCS}$ if and only if for all $i \in [t]$

$$v_i = \sum_{y \in \{0,1\}^{s'}} \tilde{f}_{M_i}(r,y) \cdot \tilde{z}(y) \tag{4}$$

where $\tilde{z}$ is an s'-variate multilinear polynomial such that $\tilde{z}(x) = \widetilde{(w,u,x)}(x)$ for all $x \in \{0,1\}^{s'}$.

3 Tree Folding Architecture

This section presents our tree folding architecture that enables parallel proof generation for high-throughput systems. We introduce a novel approach that combines the Mangrove tree folding framework with customizable constraint systems (CCS) and operation-based chunking to achieve unprecedented scalability in SNARK proving.

3.1 Overview and Architecture

Traditional SNARK provers operate sequentially, creating a fundamental bottleneck for high-throughput applications. Our tree folding architecture addresses this limitation by organizing computations into a hierarchical tree structure where:

- **Leaf nodes** contain application-specific circuits (e.g., SHA-256, ECDSA verification)
- **Inner nodes** aggregate proofs from their children through folding operations
- **The root** produces a single proof attesting to the correctness of all leaf computations

This approach enables *distributed proving* where different nodes can be processed in parallel across multiple machines, dramatically reducing end-to-end proving time for large circuits.

3.1.1 Key Architectural Components

Our system consists of four main components that work together to enable efficient tree folding:

1. **Operation-Based Chunking**: Groups similar cryptographic operations to minimize inter-chunk constraints
2. **Hierarchical Commitment Structure**: Manages witness and parameter commitments across the tree
3. **Permutation Consistency**: Ensures global constraint satisfaction through randomized permutation arguments
4. **Folding Algorithms**: Combines proofs at each tree level while preserving correctness

3.1.2 Section Organization

This section is organized as follows:

- **Section 3.2**: Details the commitment structure and constraint management strategy
- **Section 3.3**: Describes leaf node processing and CCS linearization
- **Section 3.4**: Presents the inner node folding algorithm
- **Section 3.5**: Analyzes performance characteristics and optimizations

3.2 Commitment Structure and Constraint Management

The tree folding architecture requires careful management of cryptographic commitments and constraint relationships. We employ a two-layer commitment structure combined with operation-based chunking to optimize both proving efficiency and constraint locality.

3.2.1 Operation-Based Chunking Strategy

Unlike uniform chunking approaches, our system organizes computations based on cryptographic operation types. This strategy dramatically reduces the number of copy constraints needed between chunks.

Chunking Organization. We partition the overall computation into operation-specific chunks:

$$\mathcal{C}_{\text{SHA}} : \text{Multiple SHA-256 hash computations} \tag{5}$$

$$\mathcal{C}_{\text{ECDSA}} : \text{Multiple ECDSA signature verifications} \tag{6}$$

$$\mathcal{C}_{\text{AGG}} : \text{Aggregation of outputs from operation chunks} \tag{7}$$

This organization provides three key benefits:

- **Constraint locality**: Similar operations share lookup tables and sub-circuits
- **Reduced copy constraints**: Only input/output connections between chunks require copy constraints
- **Natural parallelism**: Different operation types can be processed independently

For a system processing approximately 500,000 total constraints, this approach reduces copy constraint overhead by over 80% compared to uniform chunking.

3.2.2 Two-Layer Commitment Structure

Our commitment scheme employs complementary cryptographic primitives to balance efficiency and functionality:

Layer 1: Pedersen Commitments. For each chunk j, we create homomorphic commitments to witness values:

$$C_j = \text{Pedersen}(w_j; r_j) \tag{8}$$

where w_j represents the witness vector for chunk j and r_j is the commitment randomness.

Layer 2: Poseidon Merkle Tree. The Pedersen commitments are organized into an efficient Merkle tree:

$$h_w = \text{MerkleTree}_{\text{Poseidon}}([C_1, C_2, \ldots, C_T]) \tag{9}$$

This hybrid approach leverages the complementary strengths of each primitive:

- **Pedersen commitments** enable efficient homomorphic folding operations
- **Poseidon hashing** provides SNARK-friendly Merkle tree authentication

Public Parameter Commitments. Similarly, we maintain commitments to the permutation structure:

$$\text{plk}_j = \text{Poseidon}(j \| \sigma_j) \tag{10}$$

$$h_{\text{plk}} = \text{MerkleTree}_{\text{Poseidon}}([\text{plk}_1, \text{plk}_2, \ldots, \text{plk}_T]) \tag{11}$$

where σ_j encodes the copy constraints for chunk j.

3.2.3 Global Permutation Consistency

Maintaining correctness across the distributed tree structure requires a global permutation argument that ensures all copy constraints are satisfied.

Challenge Generation. After computing all commitments, we derive global challenges using the Fiat-Shamir transform:

$$(\alpha, \beta) = \mathcal{H}(h_{\text{plk}} \| h_w \| \text{public_inputs}) \tag{12}$$

Partial Products. Each chunk j computes a partial permutation product:

$$p_j = \prod_{i \in I_j} \frac{H_{\alpha,\beta}(w_i, \text{pos}(i))}{H_{\alpha,\beta}(w_i, \sigma(i))} \tag{13}$$

where $H_{\alpha,\beta}(x, y) = x + \alpha \cdot y + \beta$ and $\text{pos}(i)$, $\sigma(i)$ represent the original and permuted positions.

Global Verification. The correctness of all permutation constraints is verified by checking:

$$\prod_{j=1}^{T} p_j = 1 \tag{14}$$

This distributed approach enables efficient verification while maintaining the security guarantees of the global permutation argument.

3.3 Leaf Node Processing and CCS Linearization

Leaf nodes in our tree structure process the actual application circuits (e.g., cryptographic operations) and transform them into a linearized form suitable for folding. This section details the `LeafLinearizeCCS` algorithm that handles this transformation.

3.3.1 Algorithm Overview

The leaf linearization process converts a fresh CCS (Customizable Constraint System) instance into LCCS (Linearized CCS) format through three main steps:

1. **CCS Linearization**: Transform constraints using sum-check protocol
2. **Augmented Circuit Construction**: Build verification circuit for the linearization
3. **Witness Synthesis**: Generate complete witness for the next tree level

Function Interface.

```
LeafLinearizeCCS(
    pk_NIFS,              // NIFS prover key for folding operations
    s_CCS,                // Fixed CCS structure parameters
    u_CCS, w_base         // Fresh CCS instance and witness
) -> (
    null, null,           // Empty LCCS pair (no folding at leaf level)
    u_aug, w_aug          // Augmented circuit instance and witness
)
```

Note that leaf nodes return empty LCCS pairs since no folding occurs at this level. The output $(u_{\text{aug}}, w_{\text{aug}})$ feeds into parent nodes in the tree.

3.3.2 Step 1: CCS Linearization via Sum-Check

The first step transforms the CCS constraint system into linearized form using the sum-check protocol.

Commitment and Challenge Generation. We begin by committing to the base witness and deriving protocol challenges:

$$C = \text{Commit}(w_{\text{base}}; r_C) \tag{15}$$
$$r_C \leftarrow \mathcal{H}(\text{“commit”} \| w_{\text{base}}) \tag{16}$$
$$\beta \leftarrow \mathcal{H}(\text{“beta”} \| r_C) \tag{17}$$
$$\gamma \leftarrow \mathcal{H}(\text{“gamma”} \| r_C) \tag{18}$$

Sum-Check Protocol Execution. We execute sum-check on the polynomial:

$$g(X) = \gamma \cdot \text{eq}(\beta, X) \sum_{i=1}^{q} c_i \prod_{j \in S_i} \left[\sum_{y} M_{f_j}(X, y) \cdot z_e(y) \right] \tag{19}$$

This polynomial encodes the CCS constraints:

$$F_j(w_j) = 0 \quad \text{(Application circuit)} \tag{20}$$

$$p_j = \prod_{i \in I_j} \frac{H_{\alpha,\beta}(w_i, i)}{H_{\alpha,\beta}(w_i, \sigma_i)} \quad \text{(Permutation)} \tag{21}$$

$$h_{w,j} = \text{Poseidon}(\text{Pedersen}(w_j)) \quad \text{(Witness commitment)} \tag{22}$$

$$h_{\text{plk},j} = \text{Poseidon}(j \| \sigma_j) \quad \text{(Parameter commitment)} \tag{23}$$

Output Generation. The sum-check protocol produces:

- Evaluation point r_x from the final round
- Sum-check transcript π_{SC} containing all round polynomials
- Linear values $v_j = \sum_{y \in \{0,1\}^{s'}} M_{f_j}(r_x, y) \cdot z_e(y)$ for each selector

3.3.3 Step 2: Augmented Circuit Construction

The second step builds an R1CS circuit that verifies the sum-check execution and computes auxiliary values needed for tree folding.

Circuit Public Inputs. The augmented circuit takes as public inputs:

$$(x, r_x, v_1, \ldots, v_t) \tag{24}$$

where x is the original public input and $(r_x, v_1, \ldots, v_t)$ encode the linearization.

Verification Constraints. The augmented circuit enforces three categories of constraints:

Application Circuit Verification: Re-executes the original computation to ensure consistency:

$$F_j(w_{\text{base}}) = 0 \tag{25}$$

Sum-Check Verification: Validates the sum-check transcript by checking round polynomial consistency:

$$p_k(0) + p_k(1) = p_{k-1}(r_{k-1}) \quad \text{for } k = 1, \ldots, s \tag{26}$$

Final Evaluation: Confirms the linearization is correct:

$$\sum_{i=1}^{q} c_i \prod_{j \in S_i} v_j = 0 \tag{27}$$

3.3.4 Step 3: Witness Synthesis and Output

The final step constructs the complete witness and instance for the augmented circuit.

Augmented Witness. The synthesized witness contains:

$$w_{\text{aug}} = (w_{\text{base}}, r_C, \pi_{\text{SC}}, \{v_j\}, \text{auxiliary_values}) \tag{28}$$

Instance Construction. We compute the augmented instance:

$$r_{\text{aug}} \leftarrow \mathcal{H}(\text{“aug-commit”} \| w_{\text{aug}}) \tag{29}$$

$$C_{\text{aug}} = \text{Commit}(w_{\text{aug}}; r_{\text{aug}}) \tag{30}$$

$$u_{\text{aug}} = (C_{\text{aug}}, x, r_x, v_1, \ldots, v_t) \tag{31}$$

This completes the leaf processing, producing an augmented circuit ready for folding at the parent level.

3.4 Inner Node Folding Algorithm

Inner nodes in the tree structure aggregate proofs from their children through the `FoldInnerLCCS` algorithm. This process combines multiple LCCS instances while preserving all correctness guarantees and maintaining the tree's structural integrity.

3.4.1 Algorithm Overview

The inner node folding combines four types of data from child nodes:

1. **LCCS instances** (U_1, W_1) and (U_2, W_2) from children
2. **Synthesized witnesses** (u_1, w_1) and (u_2, w_2) from previous rounds
3. **Mangrove tree data** including permutation products and commitment hashes
4. **Folding parameters** needed for homomorphic combination

Function Interface.

```
FoldInnerLCCS(
    pk_NIFS, s,                 // NIFS prover key and CCS structure
    (U_1, W_1),                 // First child LCCS instance and witness
    (U_2, W_2),                 // Second child LCCS instance and witness
    (u_1, w_1),                 // First child synthesized witness
    (u_2, w_2)                  // Second child synthesized witness
) -> (
    U_F, W_F,                   // Folded LCCS instance and witness
    u_aug, w_aug                // Augmented circuit for next level
)
```

3.4.2 Step 1: Input Validation via Sum-Check

Before folding, we must verify that both child LCCS instances are valid through a combined sum-check protocol.

Combined Sum-Check Polynomial. We define a polynomial that encodes both LCCS relations:

$$g(X) = \text{eq}(r_{x,1}, X) \cdot \left[\sum_y M_{f_j}(X, y) \cdot z_{e,1}(y)\right] + \gamma \cdot \text{eq}(r_{x,2}, X) \cdot \left[\sum_y M_{f_j}(X, y) \cdot z_{e,2}(y)\right] \tag{32}$$

where $z_{e,1}$ and $z_{e,2}$ are the witness values from the two children, and γ is a fresh random challenge.

Validation. Both LCCS instances are valid if and only if:

$$\sum_{X \in \{0,1\}^s} g(X) = 0 \tag{33}$$

The sum-check protocol produces evaluation point r'_x and auxiliary values $\sigma'_{1,j}, \sigma'_{2,j}$ needed for folding.

3.4.3 Step 2: Homomorphic Folding

Once validation is complete, we homomorphically combine the LCCS instances and synthesized witnesses.

Folding Challenge. We generate a folding challenge that incorporates all relevant data:

$$\rho \leftarrow \mathcal{H}(\text{“fold”} \| C_1 \| C_2 \| u_1 \| u_2 \| r'_x) \tag{34}$$

LCCS Folding. The LCCS components are combined using powers of ρ:

$$C_F = C_1 + \rho \cdot C_2 + \rho^2 \cdot C_{u_1} + \rho^4 \cdot C_{u_2} \tag{35}$$

$$x_F = x_1 + \rho \cdot x_2 + \rho^2 \cdot x_{u_1} + \rho^4 \cdot x_{u_2} \tag{36}$$

$$r_{x,F} = r_{x,1} + \rho \cdot r_{x,2} + \rho^2 \cdot r_{u_1} + \rho^4 \cdot r_{u_2} \tag{37}$$

$$v_{j,F} = v_{j,1} + \rho \cdot v_{j,2} + \rho^2 \cdot v_{u_1,j} + \rho^4 \cdot v_{u_2,j} \tag{38}$$

Witness Folding. Similarly, the witnesses are combined:

$$W_F = w_1 + \rho \cdot w_2 + \rho^2 \cdot w_{u_1} + \rho^4 \cdot w_{u_2} \tag{39}$$

3.4.4 Step 3: Tree Structure Aggregation

The folding process must also aggregate the Mangrove tree structure data from both children.

Permutation Product Aggregation. The partial permutation products are multiplied:

$$p_{\text{parent}} = p_1 \cdot p_2 \tag{40}$$

Commitment Hash Aggregation. The commitment hashes are combined using Poseidon:

$$h_{w,\text{parent}} = \text{Poseidon}(h_{w,1} \| h_{w,2}) \tag{41}$$

$$h_{\text{plk,parent}} = \text{Poseidon}(h_{\text{plk},1} \| h_{\text{plk},2}) \tag{42}$$

3.4.5 Step 4: Augmented Circuit Generation

The final step creates an augmented circuit that verifies the folding operation for the next tree level.

Folding Verification. The augmented circuit verifies that all folding equations were computed correctly and that the sum-check validation succeeded.

Output Generation. The algorithm returns:

- **Folded LCCS pair** (U_F, W_F) representing the combined child proofs
- **Augmented circuit** $(u_{\text{aug}}, w_{\text{aug}})$ for verification at the parent level

3.5 Performance Analysis and Optimizations

Our tree folding architecture achieves significant performance improvements through several key optimizations.

3.5.1 Complexity Analysis

Proving Time. Each folding operation requires:

- $O(t \cdot d \cdot \log m)$ field operations for sum-check protocols
- $O(n)$ group operations for commitment updates (via MSMs)
- $O(\log T)$ Poseidon hashes for tree updates

Verification Time. The final verification scales as $O(\log T)$ where T is the number of leaf chunks, compared to $O(T)$ for naive approaches.

Parallelization. The tree structure enables natural parallelization:

- All leaf nodes can be processed independently
- Inner nodes can begin processing as soon as their children complete
- Total proving time scales as $O(\log T)$ with sufficient parallelism

3.5.2 Practical Optimizations

Batched MSMs. Multiple commitment updates are batched into single multi-scalar multiplication operations, reducing elliptic curve computation overhead.

Precomputation. The public parameter commitments (Equation 10) and Merkle tree structure (Equation 11) are computed during setup, eliminating runtime overhead.

Memory Management. Intermediate values are carefully managed to minimize memory usage while maintaining verification data needed for the augmented circuits.

These optimizations enable our system to handle circuits with hundreds of thousands of constraints while maintaining practical proving times for real-world applications.

4 Efficient Circuit Primitives

Our arithmetization strategy employs several key techniques to efficiently represent cryptographic operations in rank-one constraint systems (R1CS). We focus on optimizing the two main bottlenecks in signature verification: hash computations and field arithmetic.

4.1 Dual-Form Word Representation

For efficient implementation of SHA-256, which operates on 32-bit words and performs both arithmetic and bitwise operations, we employ a dual-form representation strategy inspired by Bootle et al. [**?**].

Definition 4.1 (Dense Form) *For a b-bit word w with binary representation $w = \sum_{i=0}^{b-1} 2^i \cdot w_i$ where $w_i \in \{0, 1\}$, the dense form is:*

$$dense_b(w) = \sum_{i=0}^{b-1} 2^i \cdot w_i \in \mathbb{F}_p \tag{43}$$

Definition 4.2 (Sparse Form) *The sparse form representation interleaves zero bits between the actual bits:*

$$sparse_b(w) = \sum_{i=0}^{b-1} 4^i \cdot w_i \in \mathbb{F}_p \tag{44}$$

The sparse form enables efficient bitwise operations through the following property:

Lemma 4.3 (Bitwise Operations in Sparse Form) *For any two b-bit words u, v with sparse representations u', v':*

$$u' + v' = 2 \cdot sparse_b(u \wedge v) + sparse_b(u \oplus v) \tag{45}$$

$$u' \cdot v' = sparse_b(u \wedge v) \cdot (1 + 2 \cdot sparse_b(u \oplus v)) \tag{46}$$

4.2 Lookup Arguments for Efficient Range Checking

Traditional R1CS representations of range checks require expensive bit decomposition. For checking that $x \in [0, 2^b - 1]$, the naive approach costs b constraints. We employ Häböck's lookup argument [**?**] to reduce this to a single constraint per lookup.

Definition 4.4 (Lookup Argument) *Given a table $S = \{s^{(1)}, \ldots, s^{(N)}\} \subset (\mathbb{F}_p)^k$ and a sequence $X = (x^{(1)}, \ldots, x^{(A)})$, the lookup argument proves $X \subseteq S$. The prover sends counts $c_1, \ldots, c_N \in [A+1]$ where $c^{(i)}$ represents the number of occurrences of $s^{(i)}$ in X. The verifier samples random $\alpha, \beta \in \mathbb{F}_p$ and verifies:*

$$\sum_{i=1}^{N} \frac{c^{(i)}}{\alpha + \sum_{j=1}^{k} \beta^{j-1} s_j^{(i)}} = \sum_{i=1}^{A} \frac{1}{\alpha + \sum_{j=1}^{k} \beta^{j-1} x_j^{(i)}} \tag{47}$$

4.3 Conversion Between Forms

We use lookup tables for efficient conversion between sparse and dense forms:

Algorithm 1 Sparse-Dense Conversion via Lookups

Require: Word w in sparse form, limb size parameters $(b_0, b_1, \ldots, b_{k-1})$
Ensure: Word w in dense form
1: Decompose w into limbs: $w = \sum_{j=0}^{k-1} w_j \cdot 4^{\sum_{i=0}^{j-1} b_i}$
2: **for** $j = 0$ to $k-1$ **do**
3: Lookup $(d_j, w_j) \in DS_{b_j}$ where $DS_{b_j} = \{(\text{dense}_{b_j}(x), \text{sparse}_{b_j}(x)) : x \in [0, 2^{b_j} - 1]\}$
4: **end for**
5: **return** $\sum_{j=0}^{k-1} d_j \cdot 2^{\sum_{i=0}^{j-1} b_i}$

4.4 SHA-256 Implementation

Our SHA-256 implementation achieves significant efficiency gains through careful management of word representations:

Operation	Dense Only	Sparse Only	Dual Form
32-bit XOR	96 constraints	1 constraint	9 constraints
32-bit AND	96 constraints	1 constraint	9 constraints
32-bit ADD	1 constraint	129 constraints	7 constraints
Conversion	—	—	3 lookups

Table 1: Constraint costs for different word representations in SHA-256

For 32-bit words, we use limb decomposition $(10, 11, 11)$ which:

- Minimizes lookup table sizes ($2^{10} + 2 \times 2^{11} = 5120$ entries total)
- Aligns with rotation offsets for several SHA-256 operations
- Enables table reuse across different operations

4.5 Ed25519 Verification

For Ed25519 signatures, we face the challenge of verifying the equation $sG = R + cpk$ where operations occur in the elliptic curve group over $\mathbb{F}_{2^{255}-19}$.

4.5.1 Right-Field Arithmetic

Standard approaches emulate Ed25519's base field within the constraint field $\mathbb{F}_p$, incurring substantial overhead. We eliminate this by constructing curve T-25519 with group order $2(2^{255} - 19)$ over a 256-bit field:

Definition 4.5 (T-25519 Curve) *T-25519 is a double-odd curve defined by:*

$$y^2 = x(x^2 + ax + b) \tag{48}$$

over $\mathbb{F}_{2^{256}-451024951810263391379330922557034374877}$ with discriminant $D = -2330728$.

This construction reduces fixed-base scalar multiplication from 632k to 3.2k constraints—a 200× improvement.

4.6 Modular Arithmetic Optimization

For operations modulo large integers $q \neq p$, we employ limb-wise arithmetic with optimized range checking:

1. **Limb Representation**: Express values as $\ell = \lceil |q|/b \rceil$ limbs of b bits each
2. **Polynomial Multiplication**: Verify $f_{\vec{a}}(X) \times f_{\vec{b}}(X) = f_{\vec{c}'}(X)$ at evaluation points
3. **Carry Handling**: Ensure $c'_i + o_i = 2^b o_{i+1} + c_i$ with range checks on carries and outputs

4.7 Batched Flexible Range Checks

We formalize the optimization of range check parameters as:

Definition 4.6 (Batched Flexible Range Check Problem) *Given sequences* $(l_1, u_1, n_1), \ldots, (l_k, u_k, n_k)$ *where:*

- l_i, u_i *are bit lower/upper bounds*
- n_i *is the number of values claimed to be in* $\{0, \ldots, 2^{b_i} - 1\}$ *with* $b_i \in \{l_i, \ldots, u_i\}$

Find a subrange sizing scheme $S = S_1, \ldots, S_k$ *where each* S_i *is a multiset summing to a value in* $\{l_i, \ldots, u_i\}$ *that minimizes total constraint cost:*

$$Cost(S) = \sum_{s \in \cup_i set(S_i)} 2^s + \sum_i |S_i| \cdot n_i \tag{49}$$

4.8 Performance Summary

Our arithmetization achieves the following constraint counts:

Operation	Constraints	Improvement
SHA-256 (per block)	6,064	80% reduction
Ed25519 (fixed-base mult.)	3,200	200× reduction
Modular exponentiation	10,745	8× reduction

Table 2: Constraint counts for key operations

5 Non-Uniform Proof-Carrying Data (PCD)

6 Post-Quantum Integration

7 System Architecture and Implementation

8 Performance Analysis

Acknowledgments

This work was made possible in part by the support of the Wormhole Foundation.

References

[KS23] Abhiram Kothapalli and Srinath Setty. HyperNova: Recursive arguments for customizable constraint systems. Cryptology ePrint Archive, Paper 2023/573, 2023.

[NDC+24] Wilson Nguyen, Trisha Datta, Binyi Chen, Nirvan Tyagi, and Dan Boneh. Mangrove: A scalable framework for folding-based SNARKs. Cryptology ePrint Archive, Paper 2024/416, 2024.

[PH23] Shahar Papini and Ulrich Haböck. Improving logarithmic derivative lookups using GKR. Cryptology ePrint Archive, Paper 2023/1284, 2023.

[STW23] Srinath Setty, Justin Thaler, and Riad Wahby. Customizable constraint systems for succinct arguments. Cryptology ePrint Archive, Paper 2023/552, 2023.

[ZGGX23] Tianyu Zheng, Shang Gao, Yu Guo, and Bin Xiao. KiloNova: Non-uniform PCD with zero-knowledge property from generic folding schemes. Cryptology ePrint Archive, Paper 2023/1579, 2023.

A SHA-2 CCS Constraints

This appendix provides detailed constraint system specifications for the SHA-2 implementation described in Section 3. Each operation is presented with its arithmetic constraints, lookup constraints, and matrix representation.

$\mathbf{ROTR_i}$ $(i \leq 10)$

---

limbs, sparse $\longrightarrow$ 32 bits, sparse

$x_1, x_2, x_3 \longmapsto y$

Arithmetic constraints:

$$2^{22-2i}x_3 = 2^{22}a + b$$

$$y = 2^{42}b + 2^{44-2i}x_1 + 2^{22-2i}x_2 + a$$

Lookup constraints:

$$a \in S_{10}$$

$$b \in S_{11}$$

Matrix representation for $w = (1, x_1, x_2, x_3, a, b, y)$**:**

$$\begin{bmatrix} 0 & 0 & 0 & -2^{22-2i} & 2^{22} & 1 & 0 \\ 0 & 2^{44-2i} & 2^{22-2i} & 0 & 1 & 2^{42} & -1 \end{bmatrix}$$

$\mathbf{SHR_i}$ $(i \leq 10)$

---

limbs, sparse $\longrightarrow$ 32 bits, sparse

$$x_1, x_2, x_3 \longmapsto y$$

Arithmetic constraints:

$$2^{22-2i} x_3 = 2^{22} a + b$$

$$y = 2^{44-2i} x_1 + 2^{22-2i} x_2 + a$$

Lookup constraints:

$$a \in S_{10}$$

$$b \in S_{11}$$

Matrix representation for $w = (1, x_1, x_2, x_3, a, b, y)$**:**

$$\begin{bmatrix} 0 & 0 & 0 & -2^{22-2i} & 2^{22} & 1 & 0 \\ 0 & 2^{44-2i} & 2^{22-2i} & 0 & 1 & 0 & -1 \end{bmatrix}$$

3-ary XOR

---

32 bits sparse $\longrightarrow$ limbs, dense

$$x, y, z \longmapsto f_1', f_2', f_3'$$

Arithmetic constraints:

$$x + y + z = 2e_3 + f_3 + 2^{22}(2e_2 + f_2) + 2^{44}(2e_1 + f_1)$$

Lookup constraints:

$$e_1 \in S_{10} \quad e_2, e_3 \in S_{11}$$

$$(f_1', f_1) \in DS_{10} \quad (f_2', f_2), (f_3', f_3) \in DS_{11}$$

Matrix representation for $w = (1, x, y, z, e_1, e_2, e_3, f_1, f_2, f_3)$**:**

$$\begin{bmatrix} 0 & -1 & -1 & -1 & 2^{45} & 2^{23} & 2 & 2^{44} & 2^{22} & 1 \end{bmatrix}$$

Maj

$$\begin{array}{c}\text{limbs, sparse} \longrightarrow \text{limbs, dense} \\ x_1, x_2, x_3, \\ y_1, y_2, y_3, \longmapsto e'_1, e'_2, e'_3 \\ z_1, z_2, z_3\end{array}$$

Arithmetic constraints:

$$x_3 + y_3 + z_3 + 2^{22}(x_2 + y_2 + z_2) + 2^{44}(x_1 + y_1 + z_1) = 2e_3 + f_3 + 2^{22}(2e_2 + f_2) + 2^{44}(2e_3 + f_3)$$

Lookup constraints:

$$f_1 \in S_{10} \quad f_2, f_3 \in S_{11}$$
$$(e'_1, e_1) \in DS_{10} \quad (e'_2, e_2), (e'_3, e_3) \in DS_{11}$$

Matrix representation for $w = (1, x_1, x_2, x_3, y_1, y_2, y_3, z_1, z_2, z_3, e_1, e_2, e_3, f_1, f_2, f_3)$**:**

$$\begin{bmatrix} 0 & -2^{44} & -2^{22} & -1 & -2^{44} & -2^{22} & -1 & -2^{44} & -2^{22} & -1 & 2^{45} & 2^{23} & 2 & 2^{44} & 2^{22} & 1 \end{bmatrix}$$

2-ary add

$$\begin{array}{c}\text{limbs, dense} \longrightarrow \text{limbs, dual} \\ x'_1, x'_2, x'_3, \longmapsto z'_1, z'_2, z'_3 \\ y'_1, y'_2, y'_3 \qquad z_1, z_2, z_3\end{array}$$

Arithmetic constraints:

$$x_3 + y_3 + 2^{11}(x_2 + y_2) + 2^{22}(x_1 + y_1) = 2^{32}a + z'_3 + 2^{11}z'_2 + 2^{22}z'_1$$

Lookup constraints:

$$a \in D_{10}$$
$$(z'_1, z_1) \in DS_{10} \quad (z'_2, z_2), (z'_3, z_3) \in DS_{11}$$

Matrix representation for $w = (1, x_1, x_2, x_3, y_1, y_2, y_3, a, z_1, z_2, z_3)$**:**

$$\begin{bmatrix} 0 & -2^{22} & -2^{11} & -1 & -2^{22} & -2^{11} & -1 & 2^{32} & 2^{22} & 2^{11} & 1 \end{bmatrix}$$